

Relatório de Desenvolvimento de Middleware Multicast

Garantia de Entrega Causal em Sistemas Distribuídos

Arthur Veríssimo e Davi de Castro Machado

Disciplina de Sistemas Distribuídos

16 de junho de 2026

Resumo

Este relatório descreve o projeto e a implementação de um middleware de comunicação em grupo (multicast) focado na preservação da ordenação causal de mensagens. O sistema foi desenvolvido na linguagem Java e utiliza a teoria de Relógios Lógicos Matriciais para garantir que a percepção de causa e efeito seja idêntica para todos os nós participantes, gerenciando tanto a ordenação de entrega quanto o descarte seguro de pacotes estáveis.

1 Contexto e Introdução

Em sistemas distribuídos, a falta de um relógio físico global torna o ordenamento de eventos um desafio complexo. O uso de datagramas UDP, embora ofereça alto desempenho, não provê garantias nativas de entrega ou de ordenação. O middleware desenvolvido atua como uma camada intermediária entre a rede e a aplicação, interceptando os pacotes e gerenciando dependências causais antes de realizar a entrega efetiva (*deliver*) à interface de usuário.

2 Arquitetura e Componentes do Sistema

A arquitetura do projeto foi modularizada em três camadas principais: Aplicação, Middleware e Serviços de Rede. O código-fonte foi dividido de forma a isolar as regras de negócio causal da complexidade do envio de *sockets* UDP. A Tabela 1 detalha a responsabilidade de cada arquivo implementado.

Tabela 1: Mapeamento dos Arquivos e Componentes do Sistema

Camada	Arquivo	Responsabilidade Principal
Aplicação	Main.java	Interface do usuário. Lê entradas do terminal e implementa a entrega causal no <i>prompt</i> .
Middleware	ICausalMulticast.java	Interface que define o contrato obrigatório (<i>deliver</i>) para a aplicação cliente.
Middleware	CausalMulticast.java	Núcleo lógico. Gerencia o <i>buffer</i> , avalia a ordem causal e estabiliza a matriz global.
Rede	Message.java	Estrutura do pacote UDP, contendo <i>payload</i> , ID do remetente e matriz de estado.
Rede	DiscoveryService.java	Gerencia a descoberta de nós na topologia via UDP Multicast (IP 224.0.0.1).

3 Evolução dos Mecanismos: Vetores vs. Matrizes

Para resolver o problema da causalidade, a literatura de Sistemas Distribuídos propõe diferentes estruturas lógicas. No escopo deste projeto, analisamos duas abordagens fundamentais: a Ordenação com Vetor de Relógios e a Estabilidade com Relógios Matriciais.

A **nossa implementação em Java adota os Relógios Matriciais (Matriz de Relógios)**, pois esta estrutura encapsula a lógica dos vetores, resolvendo simultaneamente o problema da ordenação e o problema do vazamento de memória (*garbage collection*).

3.1 Ordenação Causal com Vetor de Relógios (VC1)

A abordagem clássica para garantir a causalidade utiliza um Vetor unidimensional. Conforme ilustrado na Figura 1, a informação anexada à mensagem (*piggybacked*) e a informação local consistem apenas em uma dimensão, onde cada posição representa a quantidade de mensagens conhecidas de um determinado processo.

Ordenação Causal com Vetor de Relógios Lógicos

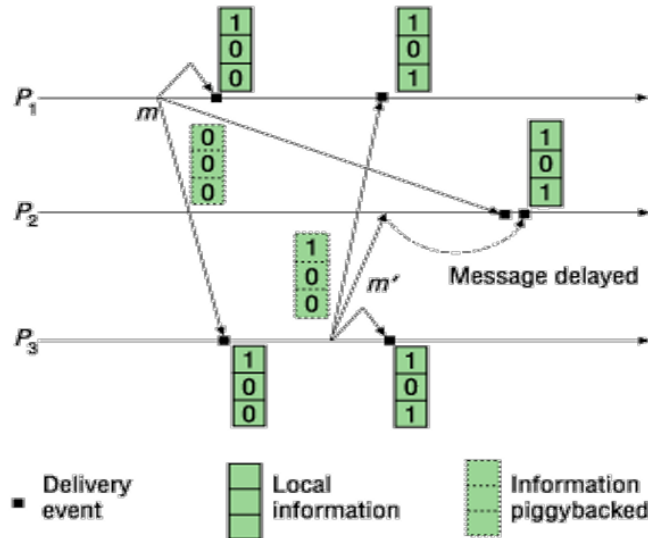


Figura 1: Ordenação Causal utilizando Vetores de Relógios Lógicos. Uma mensagem atrasada (*Message delayed*) é retida até que o vetor local do receptor alcance os pré-requisitos causais do vetor anexado.

O nosso sistema reproduz essa exata lógica na função `isCausallyOrdered()`. O middleware extrai a "linha" correspondente ao remetente dentro da matriz e a utiliza como um vetor tradicional para avaliar se a mensagem atende a duas propriedades restritas:

1. **Sequência Direta:** $\text{msg.vc}[\text{sender}][\text{sender}] == \text{mc}[\text{myId}][\text{sender}] + 1$.
2. **Passado Causal Conhecido:** $\text{msg.vc}[\text{sender}][k] \leq \text{mc}[\text{myId}][k]$.

3.2 Estabilidade com Relógios Matriciais (VC2)

Embora os vetores (Figura 1) resolvam a ordenação, eles não informam a um processo se os *outros* processos já receberam uma mensagem, impedindo o descarte seguro do pacote de memórias temporárias. Para suprir essa lacuna, a nossa implementação eleva o vetor à segunda dimensão, adotando Relógios Matriciais (mc e vc), conforme a Figura 2.

Vector Clock na Detecção de Estabilidade de Mensagens

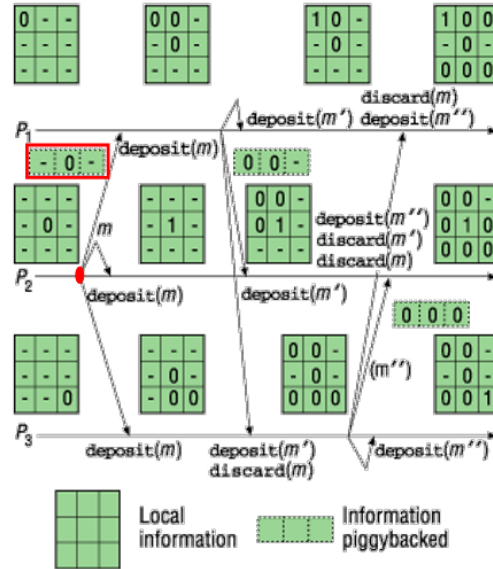


Figura 2: Diagrama demonstrando a evolução da Matriz Local e as informações anexadas às mensagens (*piggybacked*) para ordenação e descarte seguro (*discard*).

A tabela abaixo resume as estruturas adotadas no código Java com base neste modelo:

Tabela 2: Estruturas de Relógios Matriciais Implementadas

Conceito Teórico	Variável Java	Descrição e Função no Middleware
Matriz Local	mc	Representa o estado global do sistema sob a perspectiva do processo atual. Atualizada a cada evento de entrega (<i>deposit</i>).
Matriz Anexada	vc	Fotografia do estado matricial do remetente anexada ao pacote da mensagem.

Através da matriz, a operação de *discard(m)* (implementada no método `checkStabilization()`) torna-se possível. O middleware calcula o valor mínimo de uma coluna da matriz local; quando o *timestamp* da mensagem retida for menor ou igual a esse mínimo global, atesta-se que todos os nós ativos já a processaram, permitindo sua remoção segura do *buffer*.

4 Tratamento de Mensagens Atrasadas

O cenário de chegada de mensagens fora de ordem é inerente às redes não confiáveis. O middleware não descarta pacotes adiantados. Em vez disso, armazena-os em uma fila (*buffer*).

Para fins de simulação e validação da robustez, o software possui um mecanismo integrado para reter *unicasts* localmente (classe interna `DelayedUnicast`). Ao forçar a

chegada de uma resposta antes da pergunta correspondente, observou-se que o middleware isola corretamente a resposta no *buffer* e apenas a promove para a aplicação após a entrega efetiva da mensagem causalmente anterior.

5 Resiliência a Falhas Bizantinas (Mensagens Duplicadas/Divergentes)

Nosso sistema lida com anomalias de injeção paralela adotando a estratégia *first-come, first-served* em relação ao relógio lógico.

Caso um nó envie pacotes idênticos ou com conteúdos distintos portando a mesma sequência (ex: duas mensagens com ID de relógio 6), a primeira a chegar será validada, processada e incrementará o relógio local do receptor. Consequentemente, ao avaliar a segunda mensagem concorrente, a verificação causal baseada no vetor (VC1) falhará e a mensagem será retida no *buffer*. Durante a varredura global de estabilização matricial (VC2), essa mensagem anômala será identificada como obsoleta e expurgada da memória.

6 Vulnerabilidade a Falhas de Crash (Queda de Nós)

Embora o gerenciamento lógico de dependências obedeça rigorosamente à literatura acadêmica, existe um limite arquitetural mapeado referente a *crash failures* (desligamento abrupto de um nó).

Atualmente, o serviço de descoberta (*DiscoveryService*) escuta anúncios para adicionar membros à rede, porém não implementa um mecanismo de *timeout* ou *heartbeat* para remoção. Como a rotina de estabilização do *buffer* exige o cálculo do menor relógio conhecido globalmente (*mínimo da linha*), a queda de um processo congela o avanço do seu próprio vetor de estado na matriz global. Isso impossibilita a estabilização contínua das mensagens futuras, podendo acarretar saturação da memória. Como trabalho futuro, propõe-se a implementação de um descarte de instâncias baseando-se em contadores temporais de inatividade.

7 Interoperabilidade e Protocolo de Serialização na Descoberta

O enunciado do projeto estabeleceu o uso de IP Multicast para o Serviço de Descoberta, porém abstraiu a definição do protocolo de serialização da camada de rede. Diante dessa lacuna de especificação, optou-se por implementar a comunicação de descoberta transmitindo texto simples (Strings brutas em bytes) em vez de utilizar a serialização nativa de objetos do Java (através de *ObjectOutputStream*).

A escolha pelo formato de texto plano para o handshake de apresentação (transmitindo o formato padronizado "PING:porta") foi uma decisão arquitetural focada na interoperabilidade e no baixo acoplamento (*loose coupling*). A transferência de objetos serializados nativos exige que todos os nós da rede possuam a exata mesma assinatura de classe e *serialVersionUID*. Qualquer divergência estrutural em uma implementação de terceiros resultaria em exceções de desserialização, causando a rejeição silenciosa dos pacotes e isolando o processo na rede.

Ao adotar um payload de texto simples para a porta 8888 no endereço 224.0.0.1, o middleware torna-se agnóstico à implementação interna dos outros nós. O sistema apenas extrai o IP de origem do cabeçalho UDP e converte o texto recebido para identificar a porta da aplicação (Unicast), permitindo que instâncias do middleware desenvolvidas sob

diferentes arquiteturas consigam se descobrir e interagir dinamicamente na mesma rede local.

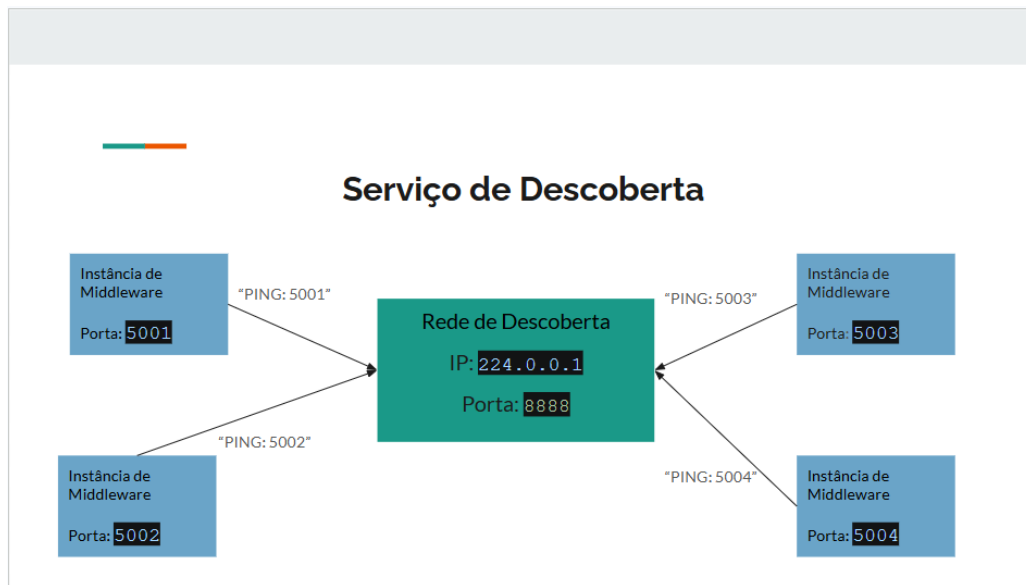


Figura 3: Diagrama demonstrando nosso serviço de descoberta.